4 Recursion

What is Recursion?

Recursion is a programming technique where a **function calls itself** repeatedly until a **base condition (terminating condition)** is met.
OR
A function solving a problem by breaking it into **smaller sub-problems of the same type**.

Basic Structure of Recursion

Every recursive function has **two main parts**:

1. **Base Case (Stopping Condition)**
   - Prevents infinite recursion
   - Terminates the function
2. **Recursive Case**
   - Function calls itself with a smaller/simpler input

Example: Print Numbers 1 to 5

```
def func(n):
    if n > 5:
        return
    print(n)
    func(n + 1)
func(1)
```

Stack Behavior

- Each recursive call is stored in the **call stack**
- If no base condition is provided → **Stack Overflow Error**

When to Use Recursion?

Use recursion when:

- The problem can be broken into **smaller sub-problems**
- Each sub-problem is **similar to the original problem**
- Solving sub-problems helps solve the main problem

Divide and Solve

Recursion works best when:

- Problem → divided into smaller pieces
- Smaller pieces → solved recursively
- Final answer → built from those solutions

Example: Factorial

Find factorial of a number `n`

$$n! = n \times (n-1) \times (n-2) \times \ldots \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Recursive case :

$$n! = n \times (n-1)!$$

Base case :

$$1! = 1, 0! = 1$$

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

What is “Recursive Leap of Faith”?

It is a mindset used while solving recursion problems:
Assume that the recursive function will correctly solve the smaller sub-problem, and focus only on how to use that result to solve the bigger problem.

Example print the sequence 321123

Identify the Sub-problem

- Break the problem into a smaller version of the same problem.
  Original: 3 2 1 1 2 3
  Sub-problem: 2 1 1 2
- Same pattern, just smaller input.

Trust / Faith

- You must **trust recursion**:
  - Assume the recursive call works correctly for smaller inputs
- Do NOT try to simulate every recursive step mentally
- Assume:
  - If the function works for smaller input, it will work for larger input

Base Condition

- Defines when recursion should stop
- If n == 0 → return

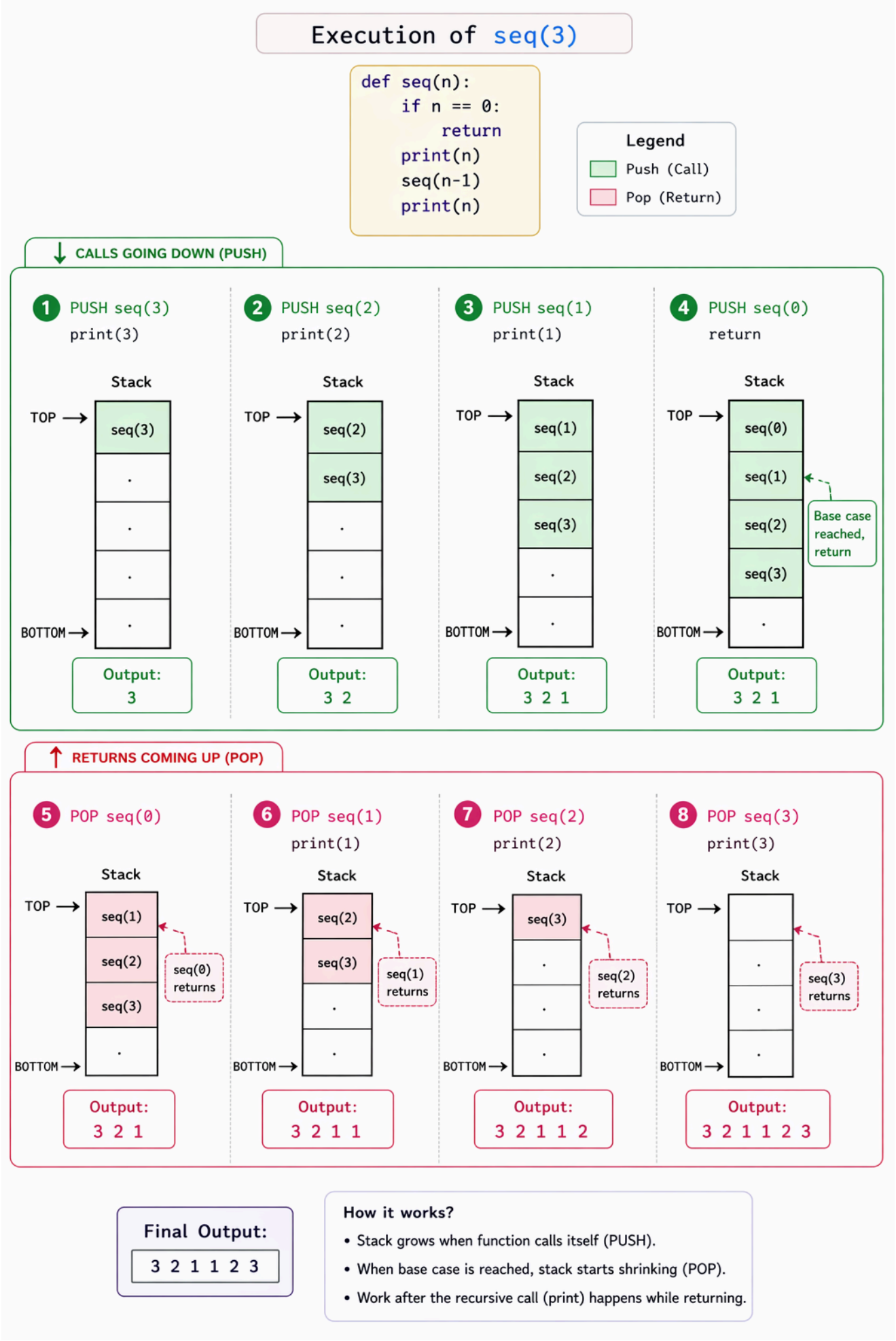
```
def seq(n):
    if n == 0:
        return
    print(n)
    seq(n - 1)
    print(n)
seq(3)
```

Visualizing Recursion

Recursion Tree

```
            seq(3)
           /      \
    print(3)     print(3)
         |
        seq(2)
       /      \
print(2)     print(2)
     |
    seq(1)
   /      \
print(1)     print(1)
 |
seq(0)
 |
   return
```

Recursion Call Stack



Ways to Write Base Condition

- Base condition tells **when to stop recursive calls**
- It prevents infinite recursion and starts **returning values**

Two Ways to Define Base Condition

1. Last Valid Input

- The last input for which recursion should still work
- After this, recursion should stop
  Example (Factorial):
- Last valid input = **1**

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$$

- So, base condition:

```
if n == 1:
    return 1
```

2. First Invalid Input

- The first input where recursion should NOT continue
- Used to stop recursion earlier

Example (Factorial):

- First invalid input = **0 (or less depending on problem)**
- So, base condition:

```
if n == 0:
    return 1
```

Example: Print n to 1

Logic:

- Keep printing until input becomes invalid

```
def print_n_to_1(n):
    if n < 1:   # First invalid input
        return
    print(n)
    print_n_to_1(n - 1)
```

- **Last valid input** (`n == 1`)

- **First invalid input** (`n < 1`)

Recurrence Relation

A **recurrence relation** expresses the solution of a problem as a function of the solutions to **smaller instances of the same problem**.

- Break a problem into **smaller subproblems**
- Solve those sub-problems
- Combine their results to get the final solution

Factorial of a Positive Integer

A recurrence relation for factorial:

$$F(n) = n \times F(n-1)$$

- To compute `F(n)`, we need the value of `F(n-1)`
- This continues until we reach the base case

How to Solve Recursion Questions

- If you can **draw the recursion tree**, you can **easily solve any recursion problem**

Example: Fibonacci Series

Sequence

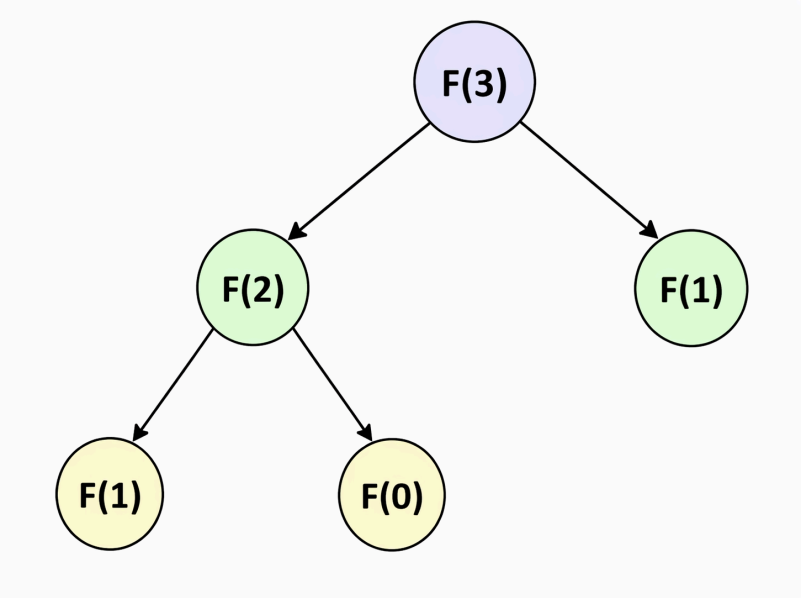
0, 1, 1, 2, 3, 5, ...

Recurrence Relation

$$F(n) = F(n-1) + F(n-2)$$

Base Conditions

F(0) = 0
F(1) = 1




```python
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

$F(2) = F(1) + F(0) = 1 + 0 = 1$
$F(3) = F(2) + F(1) = 1 + 1 = 2$
Result = > $F(3) = 2$
$O(2^n)$

```python
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

def print_series(n):
    for i in range(n):
        print(fib(i), end=" ")
```

$O(2^n)$

Aspect	Case 2	Case 1
Recursion trees	1	n
Work done	less	more
Big-O	same	same
Actual runtime	faster	slower

```python
def print_fib(n, a=0, b=1):
    if n == 0:
        return
```

```
    print(a, end=" ")
    print_fib(n-1, b, a+b)
```

$O(n)$

Recursion Approaches

You can write recursive solutions in two ways:

1. From 0 → n (Forward Recursion)

- Start from **0 (or initial value)** and move towards **n**

2. From n → 0 (Backward Recursion)

- Start from **n** and reduce towards **0**

Find Sum of First n Numbers

$$1 + 2 + 3 + \cdots + n$$

Approach 1: 0 → n

```
def sum(curr, n):
    if curr == n:
        return n
    return curr + sum(curr + 1, n)
```

$$\begin{aligned}
&sum(0, 5) \\
&= 0 + sum(1, 5) \\
&= 0 + 1 + sum(2, 5) \\
&= 0 + 1 + 2 + sum(3, 5) \\
&= 0 + 1 + 2 + 3 + sum(4, 5) \\
&= 0 + 1 + 2 + 3 + 4 + sum(5, 5) \\
&= 0 + 1 + 2 + 3 + 4 + 5 \\
&= 15
\end{aligned}$$

Approach 2: n → 0

```
def sum(n):
    if n == 0:
        return 0
    return n + sum(n - 1)
```

$$\begin{aligned}
&sum(5) \\
&= 5 + sum(4) \\
&= 5 + 4 + sum(3) \\
&= 5 + 4 + 3 + sum(2) \\
&= 5 + 4 + 3 + 2 + sum(1) \\
&= 5 + 4 + 3 + 2 + 1 + sum(0)
\end{aligned}$$

$$= 5 + 4 + 3 + 2 + 1 + 0$$
$$= 15$$